

Tipos de bases de datos NoSQL y casos de uso

Después de completar esta lectura, podrás describir las características de varios tipos de bases de datos NoSQL, enumerar sus casos de uso e identificar los proveedores frecuentemente mencionados asociados con cada tipo de base de datos NoSQL.

Al revisar cada caso de uso, verás mencionados entre paréntesis los proveedores de bases de datos NoSQL más utilizados después del caso de uso listado. Esta información no es un respaldo. Esta información es para la conciencia del mercado.

Obtén los conocimientos que necesitas para poder explicar algunas de las consideraciones técnicas asociadas al uso de MongoDB con un sistema de gestión de contenido (CMS).

Bases de datos de almacenamiento de documentos

Las bases de datos de almacenamiento de documentos, también conocidas como bases de datos orientadas a documentos, almacenan datos en un formato de documento, típicamente JSON o BSON (JSON binario), donde cada documento contiene pares de clave-valor o pares de clave-documento. Estas bases de datos no tienen esquema, lo que permite flexibilidad en las estructuras de datos dentro de una colección.

Características

- Proporciona flexibilidad de esquema: Los documentos dentro de las colecciones pueden tener estructuras variadas, lo que permite actualizaciones fáciles y la adaptación a requisitos de datos en evolución.
- Realiza operaciones eficientes de creación, lectura, actualización y eliminación (CRUD): adecuado para aplicaciones intensivas en lectura y escritura debido a su capacidad para recuperar documentos completos.
- Proporciona escalabilidad: escalabilidad horizontal mediante la fragmentación de datos en clústeres.

Casos de uso

- Sistemas de gestión de contenido (CMS): Las plataformas CMS como WordPress utilizan bases de datos de documentos para un almacenamiento y acceso rápido a tipos de contenido como artículos, imágenes y datos de usuarios. (MongoDB)
- Comercio electrónico: Las plataformas de comercio electrónico necesitan una gestión efectiva de catálogos de productos con diversos atributos y jerarquías, acomodando la naturaleza dinámica de los listados de productos en comercio electrónico. (Couchbase o Amazon DocumentDB, utilizando compatibilidad con MongoDB)

Proveedores mencionados con frecuencia

- MongoDB
- Couchbase
- Amazon DocumentDB

Almacenes de clave-valor

Los almacenes de clave-valor son las bases de datos NoSQL más simples, almacenando datos como una colección de pares de clave-valor, donde la clave es única y apunta directamente a su valor asociado.

Características

- Ofrece un alto rendimiento: eficiente para operaciones de lectura y escritura, optimizado para la recuperación rápida basada en claves
- Proporciona escalabilidad: fácilmente escalable debido a su estructura simple y capacidad para distribuir datos entre nodos
- Utiliza caché para acceso rápido
- Ofrece gestión de sesiones
- Funciona con sistemas distribuidos

Casos de uso:

- Rendimiento web mejorado mediante el almacenamiento en caché de datos de acceso frecuente (Usando Redis o Memcached)
- Plataformas de comercio electrónico, aplicaciones de software, incluyendo juegos: Amazon DynamoDB proporciona un almacén de clave-valor altamente escalable, facilitando la operación sin problemas de sistemas distribuidos al manejar alto tráfico y escalar dinámicamente.

Proveedores mencionados con frecuencia

- Redis
- Memcached
- Amazon DynamoDB

Almacenes de familias de columnas

Definición: Los almacenes de familias de columnas son bases de datos NoSQL, también conocidas como bases de datos columnares, que organizan los datos en columnas en lugar de filas. Estas bases de datos almacenan columnas de datos juntas, lo que las hace eficientes para manejar grandes conjuntos de datos con esquemas dinámicos.

Características

- Utiliza almacenamiento orientado a columnas: Los datos se agrupan por columnas en lugar de filas, lo que permite una recuperación eficiente de columnas específicas.
- Ofrece escalabilidad: Arquitectura distribuida para alta disponibilidad y escalabilidad.

Casos de uso

- Las aplicaciones de IoT gestionan grandes cantidades de datos de sensores de manera eficiente debido a su capacidad para manejar datos con marca de tiempo a gran escala, conocido como análisis de datos de series temporales. (Apache Cassandra)
- Las aplicaciones que almacenan y analizan preferencias y comportamientos de los usuarios suelen ofrecer personalización. (HBase, parte del ecosistema Hadoop)
- Análisis de datos a gran escala.

Proveedores mencionados con frecuencia

- Apache Cassandra
- HBase

Bases de datos de grafos:

Definición: Las bases de datos NoSQL de grafos están diseñadas para gestionar datos altamente interconectados, representando las relaciones como ciudadanos de primera clase junto a nodos y propiedades.

Características:

- Analiza los datos utilizando un modelo de datos de grafos: las relaciones son tan importantes como los datos en sí, lo que permite una navegación y consulta eficientes de relaciones complejas.
- Rendimiento rápido para consultas de relaciones: optimizado para consultas que involucran relaciones, lo que los hace ideales para redes sociales, sistemas de recomendación y análisis de redes.

Casos de uso:

- Las redes sociales requieren una gestión eficiente de datos sobre las relaciones entre usuarios, publicaciones, comentarios y “me gusta”. (Neo4j)
- Sistemas de recomendación: Las organizaciones necesitan una estructura de base de datos que pueda crear motores de recomendación sofisticados, analizando relaciones complejas entre usuarios, productos y comportamientos para recomendaciones precisas. (Amazon Neptune)

Proveedores mencionados con frecuencia

- Neo4j
- Amazon Neptune
- ArangoDB Memcached

Almacenes de columnas anchas:

Las bases de datos NoSQL de almacenes de columnas anchas organizan los datos en tablas, filas y columnas, como las bases de datos relacionales, pero con un esquema flexible.

Características:

- Utilizar almacenamiento en columnas: Los datos se almacenan en columnas, lo que permite una recuperación eficiente de columnas específicas en lugar de filas completas.
- Proporcionar escalabilidad horizontal y tolerancia a fallos.

Casos de uso:

- Análisis de big data: Manejo eficiente del procesamiento de datos a gran escala para análisis de big data en tiempo real. (Apache HBase utilizado junto con Hadoop)
- Gestión de contenido empresarial: Las bases de datos de grandes organizaciones necesitan gestionar vastas cantidades de datos estructurados como registros de empleados o inventarios. (Cassandra)

Proveedores mencionados con frecuencia

- Apache HBase
- Apache Cassandra

Ejemplo de caso de uso ampliado: Usando MongoDB para un sistema de gestión de contenido (CMS)

Los sistemas de gestión de contenido (CMS) recopilan, gobiernan, gestionan y enriquecen de manera inteligente el contenido empresarial, incluyendo páginas HTML, imágenes, artículos y más. Los sistemas de gestión de contenido ayudan a las empresas a desplegar su contenido de manera eficiente y segura en cualquier nube y dentro de cualquier aplicación.

Una buena gestión de contenido significa que los miembros del equipo pueden agregar, actualizar y eliminar rápidamente contenido de la base de datos y de las páginas asociadas que presentan ese contenido. Ejemplos incluyen la difusión de noticias de última hora, la actualización de noticias actuales, la inclusión de pronósticos del tiempo, la difusión de contenido publicitario, la actualización de políticas de admisión universitaria, el lanzamiento de nuevos servicios municipales, y más.

Por ejemplo, usar MongoDB como base de datos backend para un sistema de gestión de contenido (CMS) es una elección práctica cuando necesitas gestionar y servir una variedad de tipos de contenido, especialmente en escenarios donde esperas cambios frecuentes en el esquema o requisitos de escalado.

A continuación, revisemos algunos de los aspectos de la gestión de contenido utilizando un sistema de gestión de contenido, específicamente usando MongoDB.

Estructura de contenido utilizando MongoDB

En MongoDB, representas el contenido como documentos. Cada documento corresponde a una pieza de contenido, como un artículo, imagen, video o página. Puedes usar los subdocumentos en el documento para organizar la jerarquía y estructura del contenido.

Ejemplo de estructuración: Almacenamiento de una entrada de blog

Al almacenar una entrada de blog, almacenarás atributos principales como título, contenido, fecha de creación y la URL de la imagen. Luego, utilizando un campo de matriz, puedes almacenar etiquetas. Los comentarios de esa entrada se almacenan como una matriz de objetos.

```
// Collection: posts
{
  "_id":1,
  "title":"Sample Blog Post",
  "content":"This is the content of the blog post...",
  "author":{"
    "name":"John Doe",
    "email":"john@example.com",
    "bio":"A passionate blogger.",
    "created_at":"2023-09-20T00:00:00Z"
  },
  "created_at":"2023-09-20T08:00:00Z",
  "tags":["mongodb","blogging","example"],
  "comments":[
    {
      "text":"Great post!",
      "author":"Emily Johnson",
      "created_at":"2023-09-20T10:00:00Z"
    },
    {
      "text":"Thanks for sharing!",
      "author":"James Martin",
      "created_at":"2023-09-20T11:00:00Z"
    }
  ]
}
```

Metadatos e indexación usando MongoDB

Puedes utilizar las capacidades de indexación de MongoDB para optimizar la recuperación de contenido. Puedes crear índices en campos que se utilizan comúnmente para filtrar o buscar, como palabras clave, fecha de publicación o tipo de contenido, o utilizar el soporte de índices de texto de MongoDB para consultas de búsqueda de texto en campos que contienen contenido de cadena. Los índices de texto mejoran el rendimiento al buscar palabras o frases específicas dentro del contenido de cadena.

Por ejemplo, deseas proporcionar la capacidad de búsqueda en el contenido de tus blogs. Primero crearás un índice de texto:

```
db.articles.createIndex( { subject: "text" } )
```

Y luego puedes proporcionar una consulta como:

```
db.posts.find( { $text: { $search: "digital life" } } )
```

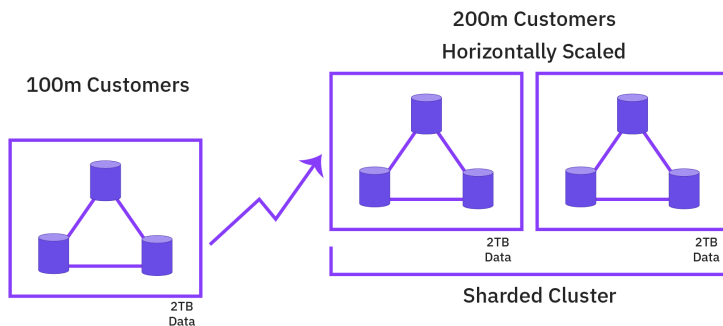
donde MongoDB buscará versiones derivadas de estas palabras: digital o life

Escalando tu CMS utilizando MongoDB

A medida que tu CMS crece, MongoDB puede ayudarte a escalar. Puedes usar sharding para escalado horizontal o usar sharding basado en zonas para distribución global.

Uso de sharding para escalado horizontal (aumento de capacidad)

Consideremos una empresa que actualmente tiene 100 millones de clientes. Esta empresa espera expandir su base de clientes a 200 millones. Este aumento en el número de clientes significa que la empresa necesitará duplicar su hardware de almacenamiento de datos de TI. La empresa puede escalar verticalmente, lo que puede costar exponencialmente más ya que el costo del hardware no es lineal con el rendimiento. El siguiente diagrama muestra que la empresa puede escalar horizontalmente y usar sharding para gestionar las bases de datos.



Sin embargo, el uso de sharding para escalado horizontal proporciona a la empresa el doble de rendimiento al doble del costo.

Resumen

Entender las características matizadas de cada tipo de base de datos NoSQL es crucial para seleccionar la solución de base de datos adecuada que se alinee con los requisitos específicos de la aplicación, las necesidades de escalabilidad y las expectativas de rendimiento.

¡Felicidades! Has completado esta lectura y estás listo para el siguiente tema.



Skills Network